

STATISTICS FOR DATA SCIENCE

(DJS23DPC253L)

Department of Computer Science and Engineering

(Data Science)

S.Y. B.Tech. Semester IV

Lab Experiment Solutions

Experiment 1A: Data Structures in Python

Experiment 1B: Measures of Central Tendency and Dispersion

Generated on: May 02, 2026

EXPERIMENT 1A: DATA STRUCTURES IN PYTHON

Date: February 3, 2026

Aim: To study data types in Python and its functions.

Software: Google Colab / Jupyter Notebook

Department of Computer Science and Engineering (Data Science) S.Y. B.Tech. Sem: IV Subject: Statistics for Data Science (DJS23DPC253L)
Experiment 1A: Data Structures in Python Date: February 3, 2026 Experiment Title: Data Structures in Python Aim: To study data types in Python and its functions. Software: Google Colab / Jupyter Notebook

Theory Data structures in Python are specialized ways of organizing and storing data so that it can be accessed, modified, and managed efficiently. Python provides built-in data structures such as: 1. Lists: Ordered and mutable collections - allow duplicate elements 2. Tuples: Ordered but immutable data - cannot be changed after creation 3. Dictionaries: Fast storage and retrieval of data using key–value pairs 4. Sets: Unordered collection of unique elements Choosing the appropriate data structure helps optimize performance, improves code readability, and enables effective problem-solving in programming.

Implementation Questions and Solutions COMMENTS

Q1. Q1. Write a comment in Python.

```
# This is a single-line comment in Python
# Comments are used to explain code and make it more readable
print("Hello, World!") # This is an inline comment
```

Q2. Q2. Write a multiline comment/paragraph in Python.

```
"""
This is a multiline comment in Python.
It can span across multiple lines.
Commonly used for docstrings to describe functions, classes, or modules.
Each line provides additional documentation.
"""
print("Multiline comments are very useful for documentation")
```

```
# Alternative method using triple single quotes
'''
This is another way to write multiline comments
using triple single quotes.
Both methods work similarly.
'''
print("Both methods work!")
```

PRIMITIVE DATA TYPES

Q3. Q3. Write a program to print an integer, float, string, complex number, Boolean, and bytes in Python and display their data type.

```
# Integer data type
integer_value = 42
print(f"Integer: {integer_value}")
print(f"Data type: {type(integer_value)}")
print()

# Float data type
float_value = 3.14159
print(f"Float: {float_value}")
print(f"Data type: {type(float_value)}")
print()

# String data type
string_value = "Hello, Python!"
print(f"String: {string_value}")
print(f"Data type: {type(string_value)}")
print()

# Complex number data type
complex_value = 3 + 4j
print(f"Complex Number: {complex_value}")
print(f"Data type: {type(complex_value)}")
print()

# Boolean data type
boolean_value = True
print(f"Boolean: {boolean_value}")
print(f"Data type: {type(boolean_value)}")
print()

# Bytes data type
bytes_value = b"Hello"
print(f"Bytes: {bytes_value}")
print(f"Data type: {type(bytes_value)}")
```

DATA STRUCTURES: LISTS

Q4. Q4. Write a program to create a list. Collect heterogenous data in it.

```
# Create a list with heterogeneous data types
heterogeneous_list = [42, 3.14, "Python", True, [1, 2, 3], {"name": "John"}]
print("Heterogeneous List:")
print(heterogeneous_list)
print(f"\nList contains: integers, floats, strings, booleans, nested lists, and dictionaries")
```

Q5. Q5. Write a program to print a list.

```
# Create and print a list
fruits = ["apple", "banana", "cherry", "date", "elderberry"]
print("Original List:")
print(fruits)
# Print each element on a new line
print("\nElements one by one:")
for fruit in fruits:
    print(fruit)
```

Q6. Q6. Write a program to print a new list. Append an item in this list.

```
# Create a new list
colors = ["red", "blue", "green"]
print("Original List:")
print(colors)
# Append a new item
colors.append("yellow")
print("\nList after appending 'yellow':")
print(colors)
```

Q7. Q7. Write a program to make a copy of the previous list.

```
# Make a copy of the list
original_list = ["red", "blue", "green", "yellow"]
print("Original List:")
print(original_list)
# Method 1: Using copy() method
copied_list1 = original_list.copy()
print("\nCopied List (using copy() method):")
print(copied_list1)
# Method 2: Using list() constructor
copied_list2 = list(original_list)
print("\nCopied List (using list() constructor):")
print(copied_list2)
# Method 3: Using slicing
```

```
copied_list3 = original_list[:]
print("\nCopied List (using slicing):")
print(copied_list3)
```

Q8. Q8. Write a program to concatenate 2 lists and print the output.

```
# Create two lists
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
print("List 1:", list1)
print("List 2:", list2)
# Method 1: Using + operator
concatenated_list1 = list1 + list2
print("\nConcatenated List (using + operator):")
print(concatenated_list1)
# Method 2: Using extend() method
concatenated_list2 = list1.copy()
concatenated_list2.extend(list2)
print("\nConcatenated List (using extend()):")
print(concatenated_list2)
```

Q9. Q9. Write a program to count the number of elements present in a list.

```
# Create a list and count elements
numbers = [10, 20, 30, 40, 50, 60, 70]
print("List:", numbers)
# Count specific element
count_30 = numbers.count(30)
print(f"Count of 30: {count_30}")
# Count with duplicate values
list_with_duplicates = [5, 5, 5, 10, 15, 5, 20]
print(f"\nList with duplicates: {list_with_duplicates}")
count_5 = list_with_duplicates.count(5)
print(f"Count of 5: {count_5}")
```

Q10. Q10. Write a program to print the length of a list.

```
# Print length of different lists
fruits = ["apple", "banana", "cherry", "date"]
print(f"Fruits list: {fruits}")
print(f"Length: {len(fruits)}")
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(f"\nNumbers list: {numbers}")
print(f"Length: {len(numbers)}")
empty_list = []
print(f"\nEmpty list: {empty_list}")
```

```
print(f"Length: {len(empty_list)}")
```

Q11. Q11. Write a program to append more than 1 item in a list.

```
# Create a list and append multiple items
my_list = [1, 2, 3]
print("Original list:", my_list)
# Append multiple items one by one
my_list.append(4)
my_list.append(5)
my_list.append(6)
print("\nAfter appending 4, 5, 6:")
print(my_list)
# Alternative: Using extend to add multiple items at once
my_list.extend([7, 8, 9, 10])
print("\nAfter extending with [7, 8, 9, 10]:")
print(my_list)
```

Q12. Q12. Write a program to extend a list.

```
# Create a list and extend it
list1 = ["a", "b", "c"]
list2 = ["d", "e", "f"]
list3 = ["g", "h", "i"]
print("List 1:", list1)
print("List 2:", list2)
print("List 3:", list3)
# Extend list1 with list2
list1.extend(list2)
print("\nAfter extending list1 with list2:")
print("List 1:", list1)
# Extend list1 with list3
list1.extend(list3)
print("\nAfter extending list1 with list3:")
print("List 1:", list1)
```

Q13. Q13. Write a program to insert a value at a position in a list.

```
# Create a list and insert values at specific positions
numbers = [10, 20, 30, 40, 50]
print("Original list:", numbers)
# Insert 15 at index 1
numbers.insert(1, 15)
print("\nAfter inserting 15 at index 1:")
print(numbers)
# Insert 35 at index 4
```

```

numbers.insert(4, 35)
print("\nAfter inserting 35 at index 4:")
print(numbers)
# Insert at beginning (index 0)
numbers.insert(0, 5)
print("\nAfter inserting 5 at index 0:")
print(numbers)

```

Q14. Q14. Write a program to delete a value at a given position in a list.

```

# Create a list and delete values at specific positions
colors = ["red", "blue", "green", "yellow", "purple"]
print("Original list:", colors)
# Delete element at index 2
del colors[2]
print("\nAfter deleting element at index 2:")
print(colors)
# Delete element at index 0
del colors[0]
print("\nAfter deleting element at index 0:")
print(colors)
# Delete last element
del colors[-1]
print("\nAfter deleting last element:")
print(colors)

```

Q15. Q15. Write a program to remove a value from the list.

```

# Create a list and remove specific values
fruits = ["apple", "banana", "cherry", "banana", "date"]
print("Original list:", fruits)
# Remove first occurrence of 'banana'
fruits.remove("banana")
print("\nAfter removing 'banana':")
print(fruits)
# Remove another value
fruits.remove("cherry")
print("\nAfter removing 'cherry':")
print(fruits)
# Try removing with pop() method
numbers = [10, 20, 30, 40, 50]
print("\nNumbers list:", numbers)
removed = numbers.pop(2) # Remove and return element at index 2
print(f"Removed element: {removed}")
print("Remaining list:", numbers)

```

Q16. Q16. Write a program to slice the data in a list.

```
# Create a list and slice it
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print("Original list:", numbers)

# Get first 5 elements
print("\nFirst 5 elements (numbers[:5]):", numbers[:5])

# Get elements from index 2 to 7 (exclusive of 7)
print("Elements from index 2 to 7 (numbers[2:7]):", numbers[2:7])

# Get elements with step
print("Every 2nd element (numbers[::2]):", numbers[::2])

# Get elements in reverse
print("Reversed list (numbers[::-1]):", numbers[::-1])
```

Q17. Q17. Write a program to slice data in a list using positions.

```
# Create a list and demonstrate different slicing operations
items = ["item0", "item1", "item2", "item3", "item4", "item5", "item6", "item7"]
print("Original list:", items)
print("\nSlicing operations:")

# From index 1 to 4
print(f"items[1:4]: {items[1:4]}")

# From index 0 to 3
print(f"items[0:3]: {items[0:3]}")

# From index 4 onwards
print(f"items[4:]: {items[4:]}")

# From beginning to index 5
print(f"items[:5]: {items[:5]}")

# With step
print(f"items[1:7:2]: {items[1:7:2]}")
```

Q18. Q18. Write a program to print the last 8 elements.

```
# Create a list with at least 8 elements
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110]
print("Original list:", numbers)
print(f"Length: {len(numbers)}")

# Print last 8 elements using negative indexing
print("\nLast 8 elements (numbers[-8:]):")
print(numbers[-8:])

# Alternative method
print("\nUsing len():")
print(numbers[len(numbers)-8:])
```

Q19. Q19. Write a program to print the last value of a list.

```
# Create a list and print the last value
```

```

numbers = [10, 20, 30, 40, 50]
print("List:", numbers)
# Method 1: Using index -1
print(f"\nLast value (numbers[-1]): {numbers[-1]}")
# Method 2: Using len()
print(f"Last value (numbers[len(numbers)-1]): {numbers[len(numbers)-1]}")
# Method 3: Using pop() - removes the element
numbers_copy = numbers.copy()
last = numbers_copy.pop()
print(f"Last value (using pop()): {last}")
print(f"List after pop(): {numbers_copy}")

```

Q20. Q20. Write a program to print the central value of a list.

```

# Create a list and print the central/middle value
numbers = [10, 20, 30, 40, 50]
print("List with odd length:", numbers)
middle_index = len(numbers) // 2
print(f"Central value: {numbers[middle_index]}")
# For list with even length
numbers_even = [10, 20, 30, 40, 50, 60]
print(f"\nList with even length: {numbers_even}")
middle_index_even = len(numbers_even) // 2
print(f"Middle indices: {middle_index_even-1} and {middle_index_even}")
print(f"Values at middle: {numbers_even[middle_index_even-1]} and {numbers_even[middle_index_even]}")
print(f"Average of middle values: {(numbers_even[middle_index_even-1] + numbers_even[middle_index_even]) / 2}")

```

DATA STRUCTURES: TUPLES

Q21. Q21. Write a program to create a tuple. Collect heterogenous data in it.

```

# Create a tuple with heterogeneous data types
heterogeneous_tuple = (42, 3.14, "Python", True, [1, 2, 3], {"name": "John"})
print("Heterogeneous Tuple:")
print(heterogeneous_tuple)
print(f"\nTuple length: {len(heterogeneous_tuple)}")
print(f"Tuple is immutable - once created, it cannot be modified")

```

Q22. Q22. Write a program to print the position of an item in the tuple.

```

# Create a tuple and find position of items
fruits = ("apple", "banana", "cherry", "date", "elderberry")
print("Tuple:", fruits)
# Find position using index()

```

```

position_banana = fruits.index("banana")
print(f"\nPosition of 'banana': {position_banana}")
position_cherry = fruits.index("cherry")
print(f"Position of 'cherry': {position_cherry}")
position_apple = fruits.index("apple")
print(f"Position of 'apple': {position_apple}")

```

Q23. Q23. Print a new tuple. Write a program to concatenate two tuples.

```

# Create two tuples
tuple1 = (1, 2, 3, 4)
tuple2 = (5, 6, 7, 8)
print("Tuple 1:", tuple1)
print("Tuple 2:", tuple2)
# Concatenate tuples using + operator
concatenated_tuple = tuple1 + tuple2
print("\nConcatenated Tuple:")
print(concatenated_tuple)

```

Q24. Q24. Write a program to print the value at position 2 in the concatenated tuple.

```

# Create and concatenate tuples
tuple1 = (10, 20, 30, 40)
tuple2 = (50, 60, 70, 80)
concatenated_tuple = tuple1 + tuple2
print("Tuple 1:", tuple1)
print("Tuple 2:", tuple2)
print("\nConcatenated Tuple:", concatenated_tuple)
# Print value at position/index 2
print(f"\nValue at index 2: {concatenated_tuple[2]}")
print(f"Value at index 0: {concatenated_tuple[0]}")
print(f"Value at index 5: {concatenated_tuple[5]}")

```

Q25. Q25. Write a program to change the element of a tuple.

```

# Note: Tuples are immutable, so we cannot change elements directly
# But we can convert to list, modify, and convert back
original_tuple = (10, 20, 30, 40, 50)
print("Original Tuple:", original_tuple)
# Method 1: Convert to list, modify, convert back to tuple
temp_list = list(original_tuple)
temp_list[2] = 99 # Change element at index 2
modified_tuple = tuple(temp_list)
print("\nModified Tuple (changed index 2 to 99):", modified_tuple)
# Method 2: Create a new tuple with modified element
new_tuple = original_tuple[:2] + (100,) + original_tuple[3:]

```

```
print("\nNew Tuple (changed index 2 to 100):", new_tuple)
```

DATA STRUCTURES: DICTIONARY

Q26. Q26. Write a program to create and print a dictionary.

```
# Create and print a dictionary

student = {
    "name": "John Doe",
    "roll_number": 101,
    "email": "john@example.com",
    "age": 20,
    "gpa": 3.8
}

print("Student Dictionary:")
print(student)

# Print formatted
print("\nFormatted Output:")
for key, value in student.items():
    print(f"{key}: {value}")
```

Q27. Q27. Write a program to print values of a dictionary using keys.

```
# Create a dictionary

person = {
    "name": "Alice",
    "age": 25,
    "city": "New York",
    "profession": "Engineer"
}

print("Dictionary:", person)

# Access values using keys
print(f"\nName: {person['name']}")
print(f"Age: {person['age']}")
print(f"City: {person['city']}")
print(f"Profession: {person['profession']}")

# Using get() method
print(f"\nUsing get() method:")
print(f"Age: {person.get('age')}")
print(f"Salary (not in dict): {person.get('salary', 'Not available')}")
```

Q28. Q28. Write a program to create a multidimensional dictionary.

```
# Create a multidimensional (nested) dictionary

company = {
    "employee1": {
```

```

"name": "John",
"department": "IT",
"salary": 50000
},
"employee2": {
"name": "Sarah",
"department": "HR",
"salary": 45000
},
"employee3": {
"name": "Mike",
"department": "Finance",
"salary": 55000
}
}

print("Multidimensional Dictionary:")
print(company)
# Print in formatted way
print("\nFormatted Output:")
for emp_id, emp_details in company.items():
    print(f"\n{emp_id}:")
    for key, value in emp_details.items():
        print(f" {key}: {value}")

```

Q29. Q29. Write a program to print values from the multidimensional dictionary using keys.

```

# Use the multidimensional dictionary from Q28
company = {
"employee1": {
"name": "John",
"department": "IT",
"salary": 50000
},
"employee2": {
"name": "Sarah",
"department": "HR",
"salary": 45000
}
}

# Access values using nested keys
print("Accessing nested dictionary values:")
print(f"Employee 1 name: {company['employee1']['name']}")
print(f"Employee 1 department: {company['employee1']['department']}")

```

```

print(f"Employee 1 salary: {company['employee1']['salary']}")
print(f"\nEmployee 2 name: {company['employee2']['name']}")
print(f"Employee 2 department: {company['employee2']['department']}")
print(f"Employee 2 salary: {company['employee2']['salary']}")

```

CONTROL FLOW: IF-ELSE CONDITION

Q30. Q30. Brother is 12 years old. Sister is 15 years old. Write a program that prints who is older using if-else statement.

```

# Fixed ages
brother_age = 12
sister_age = 15
print(f"Brother's age: {brother_age}")
print(f"Sister's age: {sister_age}")
print()
# Using if-else statement
if brother_age > sister_age:
    print("Brother is older")
elif sister_age > brother_age:
    print("Sister is older")
else:
    print("Both are of the same age")

```

Q31. Q31. Take the input of ages from the user. Write a program that prints who is older using if-else statement.

```

# Take input from user
brother_age = int(input("Enter brother's age: "))
sister_age = int(input("Enter sister's age: "))
print(f"\nBrother's age: {brother_age}")
print(f"Sister's age: {sister_age}")
print()
# Using if-else statement
if brother_age > sister_age:
    print(f"Brother is older by {brother_age - sister_age} year(s)")
elif sister_age > brother_age:
    print(f"Sister is older by {sister_age - brother_age} year(s)")
else:
    print("Both are of the same age")

```

LOOPS: FOR LOOP

Q32. Q32. Write a program that prints the elements of a list using for loop.

```

# Create a list

```

```

fruits = ["apple", "banana", "cherry", "date", "elderberry"]
print("List of fruits:")
print(fruits)
print("\nElements using for loop:")
# Print elements using for loop
for fruit in fruits:
    print(fruit)

```

Q33. Q33. Write a program that enumerates and prints the elements of a list using for loop.

```

# Create a list
colors = ["red", "green", "blue", "yellow", "purple"]
print("List of colors:")
print(colors)
print("\nEnumerated elements:")
# Print with enumeration
for index, color in enumerate(colors):
    print(f"Index {index}: {color}")

```

FUNCTIONS

Q34. Q34. Write a program to create a function.

```

# Create a simple function
def greet():
    """A simple function that greets the user"""
    print("Hello! Welcome to Python Functions!")
    print("This is a simple function without parameters.")
# Call the function
greet()

```

Q35. Q35. Create a function that adds two numbers.

```

# Create a function that adds two numbers
def add(a, b):
    """Function to add two numbers and return the result"""
    result = a + b
    return result
# Call the function with different numbers
print("Function to add two numbers:")
print(f"add(10, 20) = {add(10, 20)}")
print(f"add(5, 15) = {add(5, 15)}")
print(f"add(100, 200) = {add(100, 200)}")

```

Q36. Q36. Create a function that adds two numbers. Take input from the user.

```

# Create a function that adds two numbers

```

```
def add_numbers():
    """Function to take input from user and add two numbers"""
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))
    sum_result = num1 + num2
    print(f"\nSum of {num1} and {num2} is: {sum_result}")
    # Call the function
    add_numbers()
```

Q37. Q37. Create a function that adds two strings. Take input from the user.

```
# Create a function that concatenates two strings
def add_strings():
    """Function to concatenate two strings from user input"""
    str1 = input("Enter first string: ")
    str2 = input("Enter second string: ")
    concatenated = str1 + str2
    print(f"\nConcatenation of '{str1}' and '{str2}' is: '{concatenated}'")
    # Call the function
    add_strings()
```

DATA STRUCTURES: SETS

Q38. Q38. Write a program to create and print a set.

```
# Create and print a set
fruits = {"apple", "banana", "cherry", "date", "elderberry"}
print("Set of fruits:")
print(fruits)
print(f"\nSet data type: {type(fruits)}")
print(f"Set length: {len(fruits)}")
print("Sets are unordered collections of unique elements.")
```

Q39. Q39. Write a program to print a set with duplicate values.

```
# Create a set with duplicate values
# Note: Sets automatically remove duplicates
numbers = {1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5}
print("Set created with duplicate values:")
print(f"Original input: {{1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5}}")
print(f"\nSet output (duplicates removed):")
print(numbers)
print(f"\nNote: Sets automatically eliminate duplicate elements")
```

Q40. Q40. Write a program to print the length of a set.

```
# Create sets and print their lengths
```

```

set1 = {10, 20, 30, 40, 50}
print(f"Set 1: {set1}")
print(f"Length of Set 1: {len(set1)}")
set2 = {"a", "b", "c", "d", "e", "f", "g"}
print(f"\nSet 2: {set2}")
print(f"Length of Set 2: {len(set2)}")
empty_set = set() # Create an empty set
print(f"\nEmpty Set: {empty_set}")
print(f"Length of Empty Set: {len(empty_set)}")

```

Q41. Q41. Write a program to create a set and print its data type.

```

# Create a set and print its data type
my_set = {42, 3.14, "hello", (1, 2)}
print(f"Set: {my_set}")
print(f>Data type: {type(my_set)}")
print(f>Type name: {type(my_set).__name__}")
# Check data type of different collections
print("\nComparison of data types:")
print(f"List [1, 2, 3] type: {type([1, 2, 3])}")
print(f"Tuple (1, 2, 3) type: {type((1, 2, 3))}")
print(f"Set {{1, 2, 3}} type: {type({1, 2, 3})}")
print(f"Dictionary {{'a': 1}} type: {type({'a': 1})}")

```

Q42. Q42. Write a program to check if a set takes duplicate values with different capitalization/formatting.

```

# Check duplicate values with different capitalization
set_with_caps = {"apple", "Apple", "APPLE", "Banana", "banana", "BANANA"}
print("Set with different capitalizations:")
print(f"Input: {{'apple', 'Apple', 'APPLE', 'Banana', 'banana', 'BANANA'}}")
print(f"\nOutput: {set_with_caps}")
print(f"Length: {len(set_with_caps)}")
print("\nObservation: Strings with different capitalization are treated as different values.")

# Numbers with different types
set_numbers = {1, 1.0, 1, 2, 2.0, 2}
print(f"\nSet with numbers: {set_numbers}")
print("Note: 1 and 1.0 are considered the same in Python sets")

```

Conclusion Hence, we have studied and implemented data structures in Python. The key learnings include: 1. Comments: Used to explain code and improve readability 2. Primitive Data Types: Integer, Float, String, Complex, Boolean, Bytes 3. Lists: Ordered, mutable collections with various

operations 4. Tuples: Ordered, immutable collections 5. Dictionaries: Key-value pairs for efficient data retrieval 6. Sets: Unordered collections of unique elements 7. Control Flow: If-else statements for decision making 8. Loops: For loops for iterating over collections 9. Functions: Reusable blocks of code with parameters and return values Python's data structures provide flexible and efficient ways to organize and manipulate data for various programming tasks.

--- Name: SAP ID: Signature of Faculty:

EXPERIMENT 1B: MEASURES OF CENTRAL TENDENCY AND DISPERSION

Date: February 3, 2026

Aim: To measure central tendency and dispersion of data using Python.

Software: Google Colab / Jupyter Notebook

Department of Computer Science and Engineering (Data Science) S.Y. B.Tech. Sem: IV Subject: Statistics for Data Science (DJS23DPC253L)
Experiment 1B: Measures of Central Tendency and Dispersion Date: February 3, 2026 Experiment Title: Measures of Central Tendency and Dispersion Aim: To measure central tendency and dispersion of data using Python Software: Google Colab / Jupyter Notebook

Theory Measures of central tendency and dispersion are fundamental statistical tools used to summarize and understand data. Measures of Central Tendency: - Mean: The arithmetic average of all values - Median: The middle value when data are arranged in order - Mode: The most frequently occurring value Measures of Dispersion: - Range: The difference between maximum and minimum values - Variance: The average of squared deviations from the mean - Standard Deviation: The square root of variance - Coefficient of Variation: Ratio of standard deviation to mean - Interquartile Range: The difference between Q3 and Q1 While central tendency shows where the data are centered, dispersion reveals the degree of variability or consistency within the dataset.

Implementation Import Required Libraries

```
import numpy as np
import statistics
from collections import Counter
import matplotlib.pyplot as plt
print("Required libraries imported successfully!")
```

Q1. Find arithmetic mean of 20, 2, 7, 1, 34

```
# Q1: Find arithmetic mean
data_q1 = [20, 2, 7, 1, 34]
print(f"Data: {data_q1}")
# Method 1: Using manual calculation
mean_manual = sum(data_q1) / len(data_q1)
```

```

print(f"\nArithmetic Mean (manual): {mean_manual}")
# Method 2: Using statistics library
mean_stats = statistics.mean(data_q1)
print(f"Arithmetic Mean (statistics): {mean_stats}")
# Method 3: Using numpy
mean_numpy = np.mean(data_q1)
print(f"Arithmetic Mean (numpy): {mean_numpy}")

```

Q2. Create matrix and calculate mean for each column, row, and entire data

```

# Q2: Matrix operations
matrix = np.array([
[14, 17, 12, 33, 44],
[15, 6, 27, 8, 19],
[23, 2, 54, 1, 4]
])
print("Matrix:")
print(matrix)
# Mean of entire data
mean_all = np.mean(matrix)
print(f"\nMean of entire data: {mean_all}")
# Mean for each column
mean_columns = np.mean(matrix, axis=0)
print(f"\nMean for each column: {mean_columns}")
# Mean for each row
mean_rows = np.mean(matrix, axis=1)
print(f"\nMean for each row: {mean_rows}")

```

Q3. Find minimum, maximum value and range (column-wise and row-wise)

```

# Q3: Min, Max, Range
data_q3 = np.array([
[3, 7, 5],
[8, 4, 3],
[2, 4, 9]
])
print("Data Matrix:")
print(data_q3)
# For entire data
min_all = np.min(data_q3)
max_all = np.max(data_q3)
range_all = max_all - min_all

```

```

print(f"\nEntire Data:")
print(f"Minimum: {min_all}")
print(f"Maximum: {max_all}")
print(f"Range: {range_all}")
# Column-wise
min_cols = np.min(data_q3, axis=0)
max_cols = np.max(data_q3, axis=0)
range_cols = max_cols - min_cols
print(f"\nColumn-wise:")
print(f"Minimum: {min_cols}")
print(f"Maximum: {max_cols}")
print(f"Range: {range_cols}")
# Row-wise
min_rows = np.min(data_q3, axis=1)
max_rows = np.max(data_q3, axis=1)
range_rows = max_rows - min_rows
print(f"\nRow-wise:")
print(f"Minimum: {min_rows}")
print(f"Maximum: {max_rows}")
print(f"Range: {range_rows}")

```

Q4. Find weighted average

```

# Q4: Weighted Average
# Example: Values with corresponding weights
values = [60, 75, 85, 90]
weights = [0.2, 0.2, 0.3, 0.3] # Total should be 1
print(f"Values: {values}")
print(f"Weights: {weights}")
# Calculate weighted average
weighted_avg = np.average(values, weights=weights)
print(f"\nWeighted Average: {weighted_avg}")
# Alternative calculation
weighted_avg_manual = sum(v * w for v, w in zip(values, weights)) / sum(weights)
print(f"Weighted Average (manual): {weighted_avg_manual}")

```

Q5. Speed of 13 vehicles: Find mean, median, and mode

```

# Q5: Mean, Median, Mode
speeds = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]
print(f"Vehicle speeds: {speeds}")
print(f"Number of vehicles: {len(speeds)}")

```

```

# Mean
mean_speed = np.mean(speeds)
print(f"\nMean: {mean_speed}")

# Median
median_speed = np.median(speeds)
print(f"Median: {median_speed}")

# Mode
mode_speed = statistics.mode(speeds)
print(f"Mode: {mode_speed}")

# Also show frequency
counter = Counter(speeds)
print(f"\nFrequency of each speed:")
for speed, freq in sorted(counter.items()):
    print(f" {speed}: {freq} time(s)")

```

Q6. Calculate geometric mean for each column, row, and entire data

```

# Q6: Geometric Mean
from scipy import stats
data_q6 = np.array([
    [1, 3, 27],
    [3, 4, 6],
    [7, 6, 3],
    [3, 6, 8]
])
print("Data Matrix:")
print(data_q6)

# Geometric mean of entire data
geom_mean_all = stats.gmean(data_q6.flatten())
print(f"\nGeometric Mean (entire data): {geom_mean_all}")

# Geometric mean for each column
geom_mean_cols = np.array([stats.gmean(data_q6[:, i]) for i in range(data_q6.shape[1])])
print(f"\nGeometric Mean (each column): {geom_mean_cols}")

# Geometric mean for each row
geom_mean_rows = np.array([stats.gmean(data_q6[i, :]) for i in range(data_q6.shape[0])])
print(f"\nGeometric Mean (each row): {geom_mean_rows}")

```

Q7. Calculate harmonic mean for 1, 3, 5, 7, 9

```

# Q7: Harmonic Mean
from scipy import stats
data_q7 = [1, 3, 5, 7, 9]

```

```

print(f>Data: {data_q7}")
# Harmonic mean using scipy
harmonic_mean = stats.hmean(data_q7)
print(f"\nHarmonic Mean: {harmonic_mean}")
# Alternative calculation ( $H = n / (1/x_1 + 1/x_2 + \dots + 1/x_n)$ )
n = len(data_q7)
harmonic_mean_manual = n / sum(1/x for x in data_q7)
print(f"Harmonic Mean (manual): {harmonic_mean_manual}")

```

Q8. Calculate median for each column, row, and entire data

```

# Q8: Median for columns, rows, and entire data
data_q8 = np.array([
    [30, 65, 70],
    [80, 95, 10],
    [50, 90, 60]
])
print("Data Matrix:")
print(data_q8)
# Median of entire data
median_all = np.median(data_q8)
print(f"\nMedian (entire data): {median_all}")
# Median for each column
median_cols = np.median(data_q8, axis=0)
print(f"\nMedian (each column): {median_cols}")
# Median for each row
median_rows = np.median(data_q8, axis=1)
print(f"\nMedian (each row): {median_rows}")

```

Q9. Solar heating systems - Mean, Mode, Median analysis

```

# Q9: Solar heating systems heat-storage capacity
# Sample data for 28 systems (in days)
solar_data = [1.5, 2.0, 2.3, 2.5, 2.7, 2.8, 3.0, 3.2, 3.5, 3.8,
4.0, 4.2, 4.5, 4.8, 5.0, 5.2, 5.5, 5.8, 6.0, 6.2,
6.5, 6.8, 7.0, 7.2, 7.5, 7.8, 8.0, 8.5]
print(f"Heat-storage capacity data (n={len(solar_data)}):")
print(solar_data)
# (a) Compute the mean
mean_solar = np.mean(solar_data)
print(f"\n(a) Mean: {mean_solar}")
# (b) Compute the mode

```

```

counter_solar = Counter(solar_data)
mode_count = max(counter_solar.values())
modes_solar = [k for k, v in counter_solar.items() if v == mode_count]
print(f"\n(b) Mode: {modes_solar[0] if modes_solar else 'No mode (all values appear once)'}")

# (c) Compute the median
median_solar = np.median(solar_data)
print(f"\n(c) Median: {median_solar}")

# (d) Which measure best reflects the data?
print(f"\n(d) Analysis:")
print(f" Mean = {mean_solar:.2f}")
print(f" Median = {median_solar:.2f}")
print(f" The median is slightly less affected by extreme values and provides")
print(f" a better representation of the central tendency of this dataset.")

```

Q10. Write a function for percentile calculation

```

# Q10: Percentile Function
def calculate_percentile(data, percentile):
    """
    Calculate the percentile value of a dataset
    Args:
    data: List or array of values
    percentile: Percentile value (0-100)
    Returns:
    The value at the given percentile
    """
    sorted_data = sorted(data)
    index = (percentile / 100) * (len(sorted_data) - 1)
    # Linear interpolation
    lower_index = int(index)
    upper_index = min(lower_index + 1, len(sorted_data) - 1)
    weight = index - lower_index
    return sorted_data[lower_index] * (1 - weight) + sorted_data[upper_index] * weight

# Test data
data_q10 = [30, 40, 72, 83, 25, 10, 50, 90, 60, 15, 5, 9, 34, 23, 67, 80, 67, 45]
print(f"Data: {data_q10}")
print(f"Sorted: {sorted(data_q10)}")

# Calculate percentiles
percentiles = [30, 50, 75, 90]
print(f"\nPercentile Values:")
for p in percentiles:

```

```

value = calculate_percentile(data_q10, p)
print(f" {p}th percentile: {value:.2f}")
# Verification using numpy
print(f"\nVerification using NumPy:")
for p in percentiles:
value = np.percentile(data_q10, p)
print(f" {p}th percentile: {value:.2f}")

```

Q11. Apartment complexes - Frequency distribution and central tendency

```

# Q11: Apartment complexes analysis
apartments = [91, 79, 66, 98, 127, 139, 154, 147, 192, 88, 97, 92, 87, 142,
127, 184, 145, 162, 95, 89, 86, 98, 145, 129, 149, 158, 241]
print(f"Number of apartments in 27 complexes:")
print(apartments)
print(f"Total complexes: {len(apartments)}")
# (a) Construct frequency distribution
print(f"\n(a) Frequency Distribution:")
print(f" Interval 66-87: {sum(1 for x in apartments if 66 <= x <= 87)} complexes")
print(f" Interval 88-109: {sum(1 for x in apartments if 88 <= x <= 109)} complexes")
print(f" Interval 220-241: {sum(1 for x in apartments if 220 <= x <= 241)} complexes")
# (b) Estimate the modal value
print(f"\n(b) Modal Value:")
counter_aps = Counter(apartments)
modal_value = max(counter_aps, key=counter_aps.get)
modal_freq = counter_aps[modal_value]
print(f" Mode: {modal_value} (appears {modal_freq} times)")
# (c) Compute the mean of raw data
mean_aps = np.mean(apartments)
print(f"\n(c) Mean of raw data: {mean_aps:.2f}")
# (d) Comparison
print(f"\n(d) Comparison:")
print(f" Mode: {modal_value}")
print(f" Mean: {mean_aps:.2f}")
median_aps = np.median(apartments)
print(f" Median: {median_aps:.2f}")
print(f" The mean is a better measure because it uses all data points")
print(f" and provides a balanced representation of the central tendency.")

```

Q12. Calculate standard deviation of plant heights

```

# Q12: Standard Deviation

```

```

heights = [51, 38, 79, 46, 57] # in cm
print(f"Plant heights (cm): {heights}")

# Mean
mean_height = np.mean(heights)
print(f"\nMean: {mean_height}")

# Variance (population)
variance_pop = np.var(heights)
print(f"\nVariance (population): {variance_pop}")

# Variance (sample)
variance_sample = np.var(heights, ddof=1)
print(f"Variance (sample): {variance_sample}")

# Standard Deviation (population)
std_pop = np.std(heights)
print(f"\nStandard Deviation (population): {std_pop}")

# Standard Deviation (sample)
std_sample = np.std(heights, ddof=1)
print(f"Standard Deviation (sample): {std_sample}")

# Manual calculation
deviations = [(h - mean_height)**2 for h in heights]
std_manual = np.sqrt(sum(deviations) / (len(heights) - 1))
print(f"\nStandard Deviation (sample, manual calculation): {std_manual}")

```

Q13. Calculate range and interquartile range

```

# Q13: Range and Interquartile Range
# Sample data for 40 employees (miles driven in a year)
miles_driven = [12000, 13500, 15000, 18000, 22000, 25000, 28000, 30000, 32000, 35000,
15000, 16000, 17000, 19000, 23000, 26000, 29000, 31000, 33000, 36000,
14000, 14500, 16500, 20000, 24000, 27000, 29500, 31500, 34000, 37000,
12500, 13000, 15500, 21000, 25500, 28500, 30500, 32500, 35500, 38000]
print(f"Miles driven by {len(miles_driven)} employees:")
print(sorted(miles_driven))

# Calculate Range
min_miles = np.min(miles_driven)
max_miles = np.max(miles_driven)
range_miles = max_miles - min_miles
print(f"\nMinimum: {min_miles}")
print(f"Maximum: {max_miles}")
print(f"Range: {range_miles}")

# Calculate Quartiles
q1 = np.percentile(miles_driven, 25)
q3 = np.percentile(miles_driven, 75)

```

```

iqr = q3 - q1
print(f"\nFirst Quartile (Q1): {q1}")
print(f"Third Quartile (Q3): {q3}")
print(f"Interquartile Range (IQR): {iqr}")

```

Q14. Chef's decision on tomato weights

```

# Q14: Chef's Tomato Quality Decision
tomato_weights = [6.3, 7.2, 7.3, 8.1, 7.8, 6.8, 7.5, 7.8, 7.2, 7.5, 8.1, 8.2,
8.0, 7.4, 7.6, 7.7, 7.6, 7.4, 7.5, 8.4, 7.4, 7.6, 6.2, 7.4]
print(f"Tomato weights (ounces): {tomato_weights}")
print(f"Number of tomatoes: {len(tomato_weights)}")
# Calculate statistics
mean_weight = np.mean(tomato_weights)
std_weight = np.std(tomato_weights, ddof=1)
print(f"\nAverage weight: {mean_weight:.2f} ounces")
print(f"Standard deviation: {std_weight:.4f} ounces")
# Chef's criteria
print(f"\nChef's Criteria:")
print(f" Required average: 7.5 ounces")
print(f" Maximum acceptable std deviation: 0.5 ounce")
# Decision
print(f"\nDecision:")
if mean_weight == 7.5 and std_weight < 0.5:
    print(f" ✓ ACCEPT - Average is {mean_weight:.2f} (correct) and std dev is {std_weight:.4f} (< 0.5)")
else:
    if mean_weight != 7.5:
        print(f" ✗ REJECT - Average {mean_weight:.2f} ≠ 7.5")
    if std_weight >= 0.5:
        print(f" ✗ REJECT - Std deviation {std_weight:.4f} ≥ 0.5")

```

Q15. Compare variability between two training programs

```

# Q15: Comparison of Training Program Variability
print("Training Program Comparison:")
print("="*50)
# Group 1: Program A
mean_a = 32.11
variance_a = 68.09
std_a = np.sqrt(variance_a)
cv_a = (std_a / mean_a) * 100 # Coefficient of Variation

```

```

print(f"\nProgram A (Group 1):")
print(f" Mean training time: {mean_a} hours")
print(f" Variance: {variance_a}")
print(f" Standard Deviation: {std_a:.4f}")
print(f" Coefficient of Variation: {cv_a:.2f}%")
# Group 2: Program B
mean_b = 19.75
variance_b = 71.14
std_b = np.sqrt(variance_b)
cv_b = (std_b / mean_b) * 100
print(f"\nProgram B (Group 2):")
print(f" Mean training time: {mean_b} hours")
print(f" Variance: {variance_b}")
print(f" Standard Deviation: {std_b:.4f}")
print(f" Coefficient of Variation: {cv_b:.2f}%")
# Comparison
print(f"\n{'='*50}")
print(f"Conclusion:")
if cv_a < cv_b:
    print(f"Program A has LESS relative variability ({cv_a:.2f}% < {cv_b:.2f}%)")
    print(f"Program A is more consistent and recommended.")
else:
    print(f"Program B has LESS relative variability ({cv_b:.2f}% < {cv_a:.2f}%)")
    print(f"Program B is more consistent and recommended.")

```

Q16. Frequency distribution of ski lift users' weights (Histogram)

```

# Q16: Histogram of Ski Lift Users' Weights
# Sample frequency distribution data
weights_intervals = ['100-120', '120-140', '140-160', '160-180', '180-200']
frequencies = [15, 35, 40, 40, 20]
print("Frequency Distribution of Ski Lift Users' Weights:")
print("="*40)
for interval, freq in zip(weights_intervals, frequencies):
    print(f"{interval} lbs: {freq} people")
# Create histogram
fig, ax = plt.subplots(figsize=(10, 6))
x_positions = range(len(weights_intervals))
ax.bar(x_positions, frequencies, color='steelblue', edgecolor='black')
ax.set_xlabel('Weight Range (lbs)')
ax.set_ylabel('Frequency (Number of People)')
ax.set_title('Histogram of Ski Lift Users Weights (n=150)')

```

```

ax.set_xticks(x_positions)
ax.set_xticklabels(weights_intervals)
ax.grid(axis='y', alpha=0.3)
# Add value labels on bars
for i, v in enumerate(frequencies):
    ax.text(i, v + 1, str(v), ha='center', va='bottom')
plt.tight_layout()
plt.show()
print("\n(a) Observations from histogram:")
print(" - The distribution is approximately normal")
print(" - Most users weigh between 140-200 lbs")
print(" - Peak frequency is in the 140-160 and 160-180 ranges")
print("\n(b) Safe capacity solution:")
print(" - If chair capacity is 400 lbs for 2 people")
print(" - Most effective weight distribution across chairs needed")

```

Q17. Comparative box plot analysis of day and evening classes

```

# Q17: Box Plot Analysis

day_class = [99, 56, 78, 55.5, 32, 90, 80, 81, 56, 59, 45, 77, 84.5, 84, 70, 72, 68, 32, 79, 90]

evening_class = [98, 78, 68, 83, 81, 89, 88, 76, 65, 45, 98, 90, 80, 84.5, 85, 79, 78, 98, 90, 79, 81, 25.5]

print("Day Class Scores:")
print(sorted(day_class))
print(f"Number of students: {len(day_class)}")
print("\nEvening Class Scores:")
print(sorted(evening_class))
print(f"Number of students: {len(evening_class)}")

# Calculate statistics for Day Class
day_min = np.min(day_class)
day_max = np.max(day_class)
day_median = np.median(day_class)
day_q1 = np.percentile(day_class, 25)
day_q3 = np.percentile(day_class, 75)
print(f"\nDay Class Statistics:")
print(f" Minimum: {day_min}")
print(f" Maximum: {day_max}")
print(f" Q1 (25th percentile): {day_q1}")
print(f" Median (50th percentile): {day_median}")
print(f" Q3 (75th percentile): {day_q3}")

# Calculate statistics for Evening Class
evening_min = np.min(evening_class)

```

```

evening_max = np.max(evening_class)
evening_median = np.median(evening_class)
evening_q1 = np.percentile(evening_class, 25)
evening_q3 = np.percentile(evening_class, 75)
print(f"\nEvening Class Statistics:")
print(f" Minimum: {evening_min}")
print(f" Maximum: {evening_max}")
print(f" Q1 (25th percentile): {evening_q1}")
print(f" Median (50th percentile): {evening_median}")
print(f" Q3 (75th percentile): {evening_q3}")
# Create box plots
fig, ax = plt.subplots(figsize=(10, 6))
data_to_plot = [day_class, evening_class]
ax.boxplot(data_to_plot, labels=['Day Class', 'Evening Class'])
ax.set_ylabel('Test Scores')
ax.set_title('Comparative Box Plot: Day vs Evening Class')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
# Calculate percentage distributions
print(f"\n--- Percentage Analysis for Day Class ---")
print(f"Min to Q1: {(day_q1 - day_min) / (day_max - day_min) * 100:.1f}%")
print(f"Q1 to Median: {(day_median - day_q1) / (day_max - day_min) * 100:.1f}%")
print(f"Median to Q3: {(day_q3 - day_median) / (day_max - day_min) * 100:.1f}%")
print(f"Q3 to Max: {(day_max - day_q3) / (day_max - day_min) * 100:.1f}%")
print(f"Q1 to Q3 (middle 50%): {(day_q3 - day_q1) / (day_max - day_min) * 100:.1f}%")
print(f"\n--- Percentage Analysis for Evening Class ---")
print(f"Min to Q1: {(evening_q1 - evening_min) / (evening_max - evening_min) * 100:.1f}%")
print(f"Q1 to Median: {(evening_median - evening_q1) / (evening_max - evening_min) * 100:.1f}%")
print(f"Median to Q3: {(evening_q3 - evening_median) / (evening_max - evening_min) * 100:.1f}%")
print(f"Q3 to Max: {(evening_max - evening_q3) / (evening_max - evening_min) * 100:.1f}%")
print(f"Q1 to Q3 (middle 50%): {(evening_q3 - evening_q1) / (evening_max - evening_min) * 100:.1f}%")
# Compare spreads
day_iqr = day_q3 - day_q1
evening_iqr = evening_q3 - evening_q1
print(f"\n--- Comparison ---")
print(f"Day Class IQR: {day_iqr}")
print(f"Evening Class IQR: {evening_iqr}")
if day_iqr > evening_iqr:

```

```
print(f"Day class has WIDER spread in middle 50% (more variability)")
else:
print(f"Evening class has WIDER spread in middle 50% (more variability)")
```

Conclusion We have successfully studied and implemented measures of central tendency and dispersion in Python using various libraries (NumPy, SciPy, and Statistics). **Key takeaways:** Measures of Central Tendency: - Mean: Best for normally distributed data - Median: Best for skewed distributions or data with outliers - Mode: Best for categorical data or identifying the most frequent value Measures of Dispersion: - Range: Simple but affected by outliers - Variance: Measures average squared deviations - Standard Deviation: Square root of variance, in same units as data - Coefficient of Variation: Relative variability (useful for comparing different datasets) - Interquartile Range: Robust measure unaffected by outliers **Practical Applications:** - Understanding data distribution for better decision making - Comparing datasets with different scales - Identifying outliers and anomalies - Quality control and statistical analysis These statistical measures are fundamental tools for data analysis and are widely used in data science, business analytics, and research.

--- Name: SAP ID: Signature of Faculty:

CONCLUSION

This laboratory exercise has successfully demonstrated:

Experiment 1A - Data Structures in Python:

- Understanding of Python's built-in data structures (lists, tuples, dictionaries, sets)
- Practical application of data structure operations and manipulations
- Implementation of control flow and functions in Python
- Best practices for data organization and retrieval

Experiment 1B - Central Tendency and Dispersion:

- Calculation of statistical measures for data analysis
- Understanding of how central tendency and dispersion describe datasets
- Practical implementation using NumPy and other statistical libraries
- Application of statistical concepts to real-world data problems

Both experiments provide foundational knowledge essential for data science and statistical analysis applications.

Name: _____

SAP ID: _____

Signature of Faculty: _____